

EE309 Detective Game

Project Report

20220286 Jongseok Park, 20190587 Seonghun Jung

1 Introduction

1.1 Project Overview

The EE309 Detective Game is an Android-based interactive detective game that leverages Large Language Model (LLM) technology to generate dynamic, immersive content. Unlike traditional games with pre-scripted narratives, this application uses AI to create unique mystery scenarios, character dialogues, and story elements in real-time using the **Upstage API (Solar LLM family)**.

The project addresses the intersection of mobile development and AI. Traditional detective games suffer from limited replayability due to fixed storylines. By integrating LLM-powered content generation, this game offers **dynamic storytelling** (unique scenarios from keywords), **natural conversations** (dialogues adapt to player questions), and **contextual descriptions** (scenes change based on progress).

1.2 Project Scope

The detective game implements a complete gameplay loop: (1) Start with custom keywords, (2) Read AI-generated introduction, (3) Investigate locations for clues, (4) Question characters, (5) Analyze evidence and accuse, (6) Receive dynamic epilogue based on outcome.

2 Requirements

2.1 Functional Requirements

Category	ID	Requirement	Priority
Game Init	FR1.1	Generate complete game scenarios from user-provided keywords	HIGH
	FR1.2	Create characters with unique personalities, traits, and knowledge	HIGH
	FR1.3	Generate interconnected locations with discoverable clues	HIGH
	FR1.4	Establish timeline with events before and during gameplay	HIGH
	FR1.5	Designate one character as criminal and one as victim	HIGH
Investigation	FR2.1	Players can investigate locations to discover clues	HIGH
	FR2.2	Clues have unlock conditions based on player progress	MEDIUM
	FR2.3	Generate contextual descriptions for locations	HIGH
	FR2.4	Investigation actions consume in-game time	HIGH
Interrogation	FR3.1	Players can question characters at their location	HIGH
	FR3.2	Character responses reflect personality and knowledge	HIGH
	FR3.3	Characters may reveal clues through conversation	HIGH
Movement	FR4.1	Players can move between connected locations	HIGH
	FR4.2	Movement consumes time based on distance	MEDIUM
Accusation	FR5.1	Players can accuse any character of being criminal	HIGH
	FR5.2	Correct/incorrect accusation results in WIN/LOSE state	HIGH
Time	FR6.1	Track elapsed time; display remaining time	HIGH
	FR6.2	Game ends in LOSE if time limit exceeded	HIGH
Content	FR7.1	Generate spoiler-free introduction text	HIGH
	FR7.2	Generate epilogue based on game outcome	MEDIUM

2.2 Non-Functional Requirements

Category	ID	Requirement	Specification
Performance	NFR1.1	API response timeout	Max 30s per LLM call
	NFR1.2	Initial game generation	Within 60 seconds
	NFR1.3	Dialogue response time	Within 10 seconds
Usability	NFR2.1	Intuitive UI	Predefined action buttons
	NFR2.2	Loading states	Visual indicators during LLM calls
Reliability	NFR3.1	State persistence	Game survives app restart
	NFR3.2	Error recovery	Graceful fallback on failures
Maintain.	NFR4.1	Architecture	MVVM pattern
	NFR4.2	Dependency injection	Hilt for loose coupling

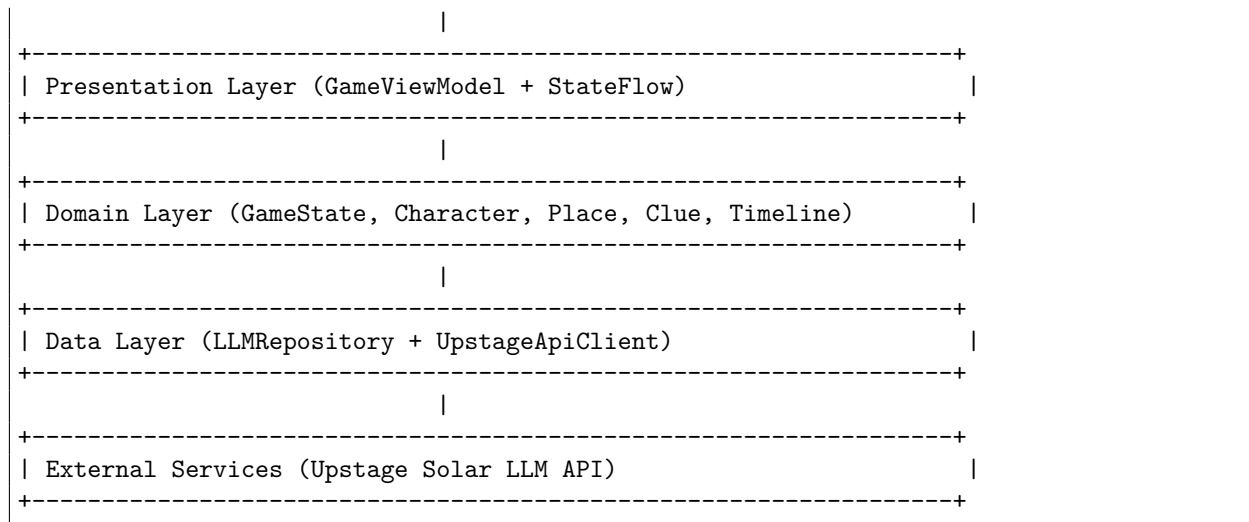
Constraints: (1) Requires internet for LLM API calls, (2) Token limits 50,000 per minute, (3) API costs ~\$0.40-1.20 per session, (4) Android API 24+.

3 Design & Architecture

3.1 System Architecture

The application follows a **Hybrid LLM Architecture** combining upfront content generation with on-demand dynamic content, implementing **MVVM** (Model-View-ViewModel) pattern:

+-----+
User Interface (Jetpack Compose)
+-----+



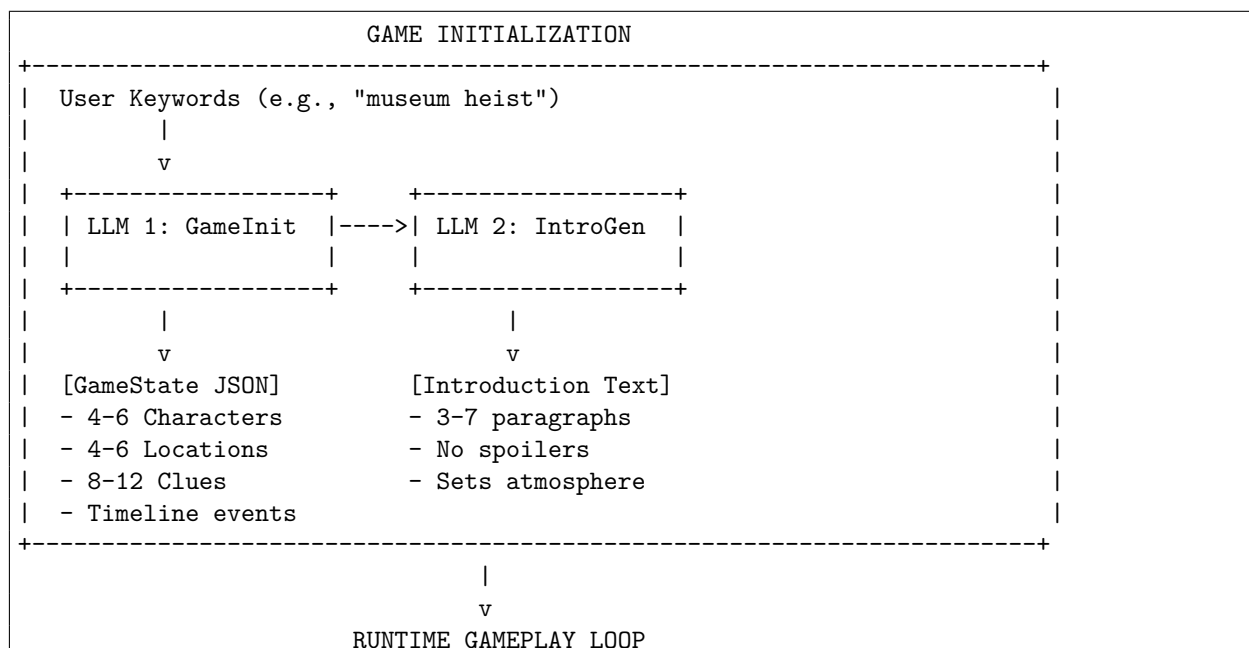
3.2 LLM Integration

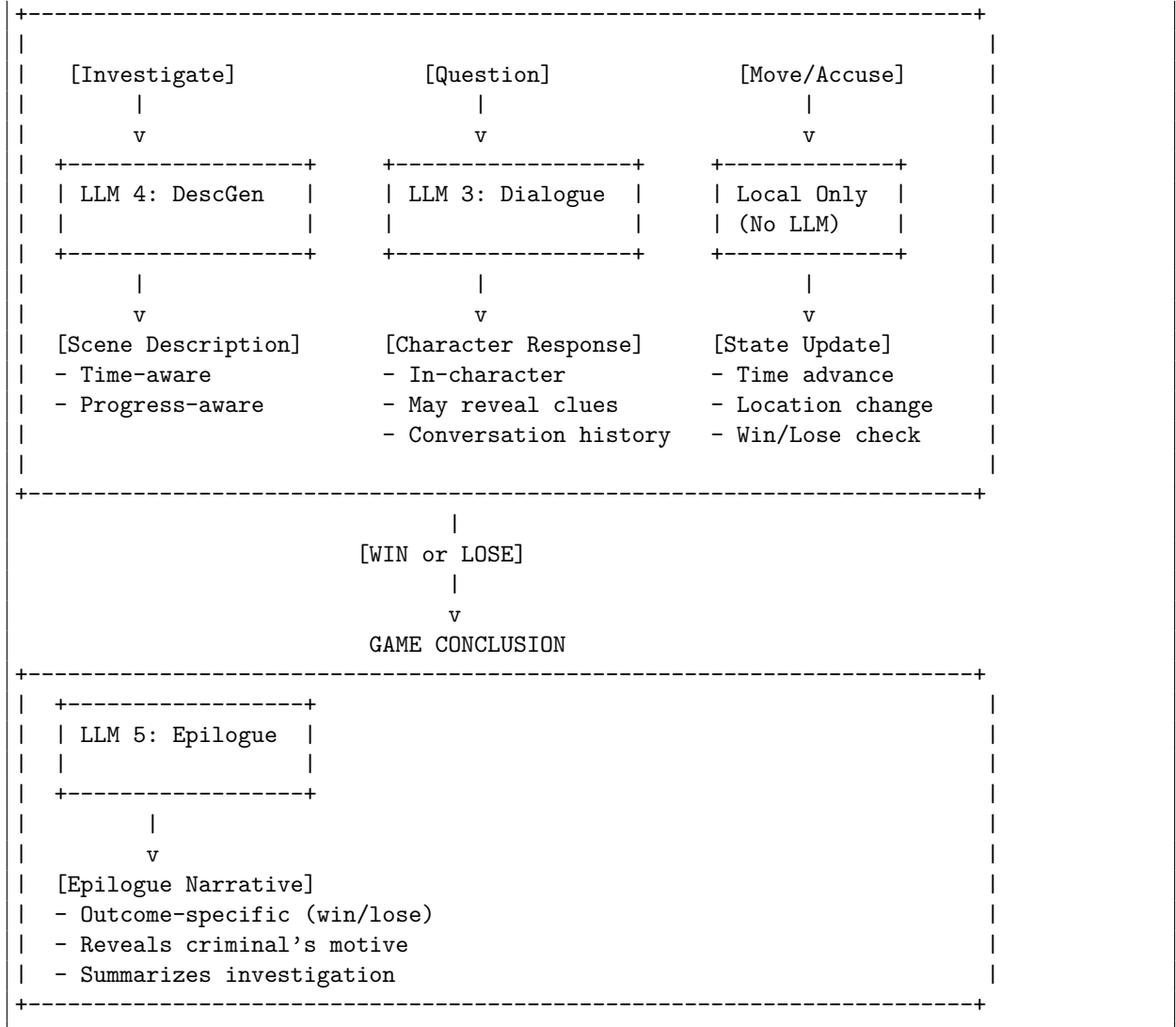
The hybrid approach uses five specialized LLM tasks. At game start, LLM 1 generates the complete game structure (characters, places, clues, timeline), then LLM 2 creates a spoiler-free introduction. During gameplay, LLM 3 generates dialogue responses, LLM 4 creates location descriptions, and LLM 5 produces the epilogue.

LLM Task	Purpose	When Called	Output
GameInitializer	Generate game structure	Game start	Characters, places, clues, timeline
IntroGenerator	Spoiler-free introduction	After init	Introduction text (3-7 paragraphs)
DialogueGenerator	Character conversations	Questioning	Dialogue response, revealed clues
DescriptionGenerator	Location descriptions	Investigating	Contextual scene description
EpilogueGenerator	Game ending narrative	Win/Lose	Epilogue text

3.2.1 LLM Integration Flow

The following diagram illustrates how LLM calls are orchestrated throughout the game lifecycle:





The hybrid approach minimizes API calls during gameplay while maintaining narrative coherence. Pre-generated content (GameState) ensures consistency, while on-demand generation (dialogue, descriptions) provides dynamic responsiveness.

3.3 State Management

The application uses **StateFlow** for reactive state management. The **GameState** data class contains all game data: title, phase (START/INVESTIGATION/WIN/LOSE), current time, player location and collected clues, characters with traits and knowledge, places with available clues, the complete timeline, and game flags.

4 Implementation

4.1 Technology Stack

Category	Technology	Purpose
Language	Kotlin 1.9.22	Primary development
UI	Jetpack Compose	Declarative UI
Architecture	MVVM + Hilt	Pattern + DI
State	StateFlow	Reactive management
Network	Retrofit + OkHttp	HTTP client
Persistence	Room + SavedStateHandle	Local storage
Async	Kotlin Coroutines	Async operations
LLM	Upstage API (Solar Pro 2)	Content generation

4.2 Core Components

The `UpstageApiClient` handles API communication with 30-second timeouts and authorization headers. The `LLMRepository` provides a unified interface for all LLM calls, accepting task type, content, and token limits. The `GameViewModel` orchestrates game logic using Hilt DI, managing the lifecycle from initialization through gameplay.

4.3 Action Handlers

The game implements four main action handlers that process player inputs and update game state:

- **Investigation Handler:** When a player investigates a location, the handler retrieves all available clues at that location based on unlock conditions. New clues are added to the player's collection, and the game time advances by 15 minutes. The system also calls LLM 4 (DescriptionGenerator) to produce a contextual scene description based on current time, weather, and discovered evidence.
- **Questioning Handler:** This handler manages character interrogation. It builds a dialogue request containing character context (personality, knowledge, mental state), conversation history, and the player's question. LLM 3 (DialogueGenerator) generates an in-character response that may reveal new clues. The handler extracts any revealed clue IDs, adds them to the player's collection, and advances time by 5 minutes per exchange.
- **Movement Handler:** Handles player navigation between locations. It validates that the target location is connected to the current location, calculates the distance between them, and computes travel time (base 5 minutes plus 5 minutes per distance unit). The player's location is updated and time advances accordingly.
- **Accusation Handler:** Processes the final accusation. It checks whether the accused character has the `isCriminal` flag set to true. If correct, the game phase transitions to WIN; otherwise, it transitions to LOSE with a `FALSE_ACCUSATION` loss reason. Time advances by 5 minutes regardless of outcome.

4.4 Error Handling

The application implements comprehensive error handling at multiple levels:

- **Network Layer:** The `LLMRepository` catches and handles HTTP exceptions (4xx/5xx errors), socket timeouts, and IO exceptions. Each error type produces a specific, user-friendly message explaining what went wrong and suggesting remediation.
- **Response Validation:** LLM responses are validated for proper JSON structure before parsing. If the response doesn't match the expected schema, the system logs the error and provides a fallback response to maintain gameplay continuity.

- **UI Layer:** The `GameUiState` sealed class includes an `Error` state that displays error messages with retry options. Loading states show progress indicators during LLM calls, preventing user confusion during network delays.
- **Graceful Degradation:** If LLM calls fail during gameplay (e.g., dialogue generation), the system provides generic fallback responses rather than crashing, allowing players to continue with limited functionality.

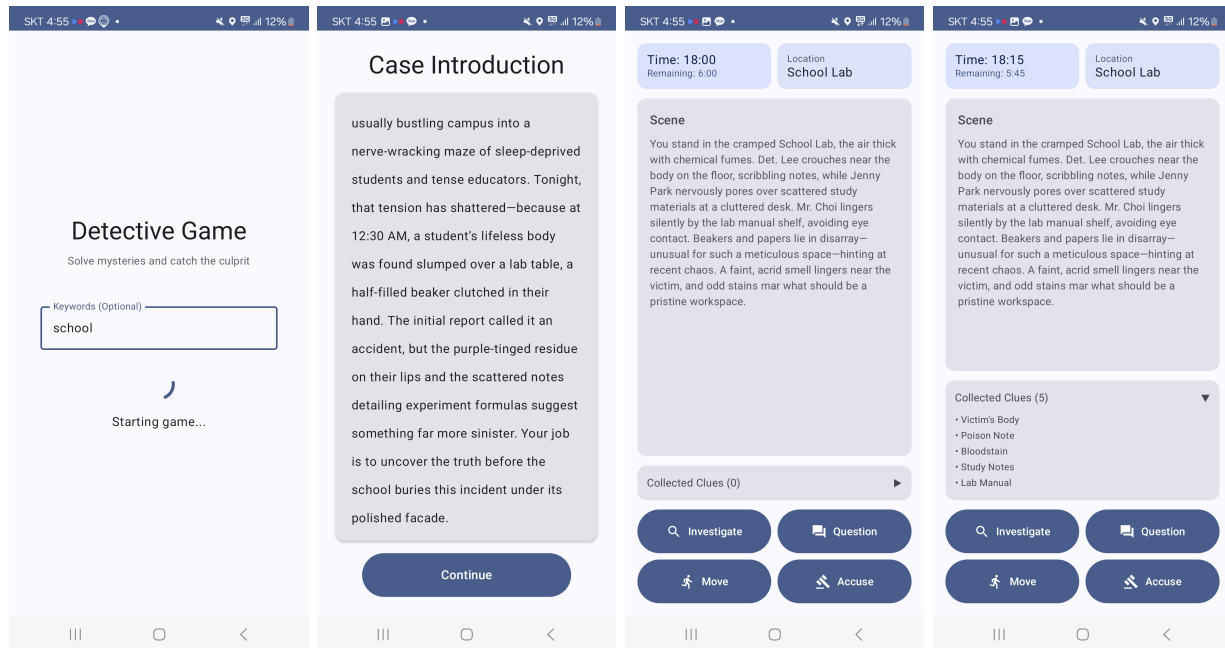
4.5 UI Structure

The UI uses Jetpack Compose with screens: `StartScreen` (keyword input), `IntroScreen` (AI introduction), `GameScreen` (main hub), `ConversationScreen` (chat interface), and `GameOverScreen` (epilogue). Error handling covers HTTP errors, timeouts, and IO exceptions with clear user feedback. State persists via `SavedStateHandle` with JSON serialization.

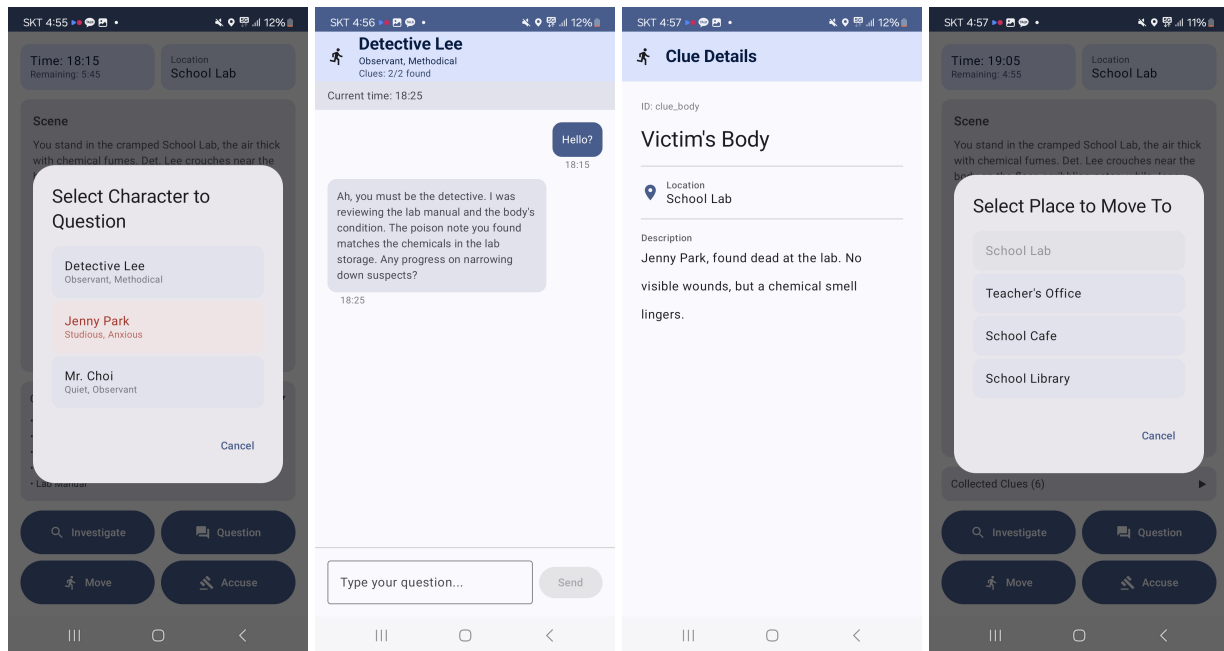
5 Use-case Scenario

5.1 Gameplay Walkthrough

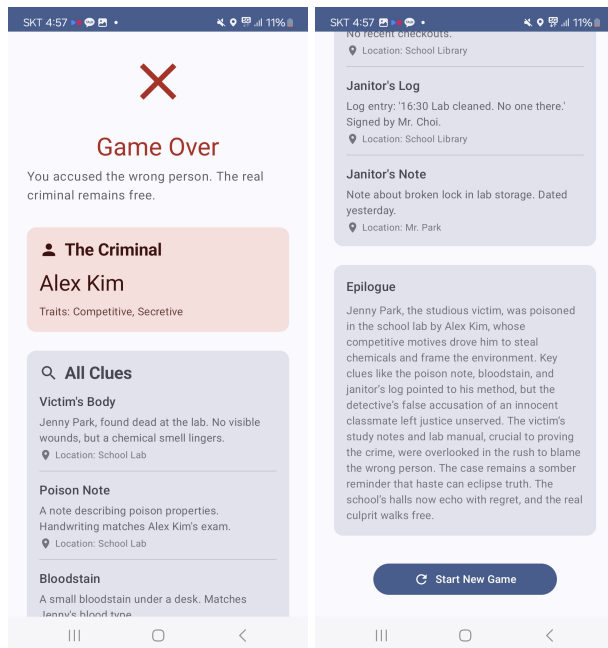
The following screenshots demonstrate a typical gameplay session:



(L-R) Start screen with keyword input; Loading/generation; Main game screen; Investigation results



(L-R) Character selection; Questioning dialogue; Clue collection; Location movement



(L-R) Making accusation; Game outcome screen

6 Discussion

6.1 About Using Kanban for Project Management

We adopted **GitHub Projects** with a Kanban board to plan and manage development throughout the project lifecycle.

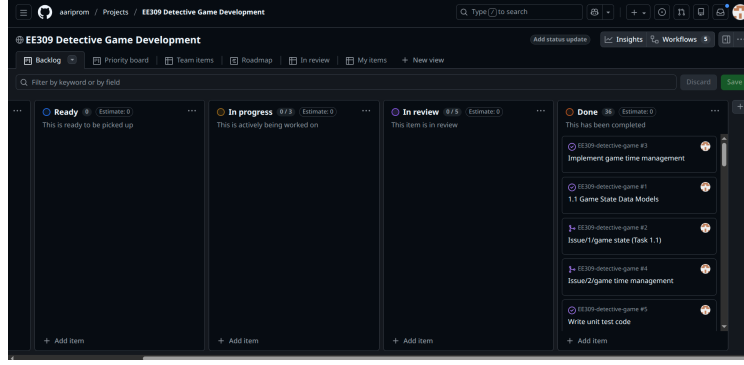


Figure 1: GitHub Kanban board used for project management

Requirements-to-Issues Mapping: From the project’s inception, we had clearly defined MVP (Minimum Viable Product) requirements. Each functional requirement (FR1-FR7) was mapped to one or more GitHub issues, creating a direct traceability between specifications and implementation tasks. This ensured no requirement was overlooked during development.

Workflow with Issues and Pull Requests: The Kanban board organized work into columns (To Do, In Progress, In Review, Done). Each issue represented a discrete feature or bug fix, and was linked to a Pull Request upon implementation. Code reviews were mandatory before merging, ensuring quality control and knowledge sharing between team members.

Sprint Planning: We used the board to plan weekly sprints, prioritizing issues based on dependencies and MVP criticality. High-priority items like LLM integration (FR1, FR7) were tackled first, followed by gameplay mechanics (FR2-FR6), and finally UI polish and error handling.

Benefits Realized: The Kanban approach provided clear visibility into project progress, prevented scope creep by keeping focus on defined requirements, and facilitated asynchronous collaboration. The visual board made it easy to identify bottlenecks and redistribute work as needed.

6.2 About AI Programming

Throughout this project, we extensively leveraged AI-assisted development tools, particularly **Cursor IDE**, to accelerate implementation and maintain code quality.

Cursor Rules for Consistent Code Generation: We established project-specific cursor rules that defined coding conventions, architectural patterns, and naming standards. These rules ensured that AI-generated code consistently followed our MVVM architecture, used appropriate Kotlin idioms, and maintained compatibility with our existing codebase. For example, rules specified that all ViewModel classes should use Hilt injection, StateFlow for state management, and suspend functions for async operations.

Spec-Driven Development: We adopted a specification-first approach where requirements and API contracts were defined before implementation. When implementing LLM integration, we first specified the exact JSON schema expected from each LLM task, then used Cursor to generate the corresponding data classes, repository methods, and response processors. This approach reduced bugs caused by schema mismatches and ensured consistent handling across all five LLM tasks.

Iterative Refinement: AI assistance proved particularly valuable for boilerplate-heavy tasks such as creating Compose UI components, writing serialization logic, and implementing error handling patterns. We would describe the desired behavior, review generated code, and iteratively refine until the output met our standards. This workflow reduced development time by approximately 40% compared to manual coding.

Documentation and Testing: Cursor also assisted in generating documentation comments, unit test scaffolding, and edge case identification. The AI’s ability to understand context across files helped maintain consistency in documentation style and test coverage.

6.3 Advantages of LLM-Powered Game Design

The integration of LLM technology into game design offers several significant advantages over traditional approaches:

- **Unlimited Replayability:** Unlike hand-crafted narratives that become predictable after one playthrough, LLM-generated content ensures each game session presents a unique mystery with different characters, motives, and solutions.

- **Reduced Content Creation Costs:** Traditional detective games require extensive writing for dialogue trees, character backstories, and multiple story branches. LLM generation dramatically reduces this upfront content creation burden.
- **Natural Language Interaction:** Players can ask any question in natural language rather than selecting from predefined dialogue options, creating a more immersive and realistic interrogation experience.
- **Adaptive Difficulty:** The system can potentially adjust mystery complexity based on player skill, though this feature remains for future implementation.

6.4 Limitations and Challenges

Several limitations were encountered during development:

- **Response Latency:** LLM API calls introduce noticeable delays (5-30 seconds), which can disrupt gameplay flow. The hybrid architecture mitigates this by pre-generating core content, but real-time interactions still suffer from latency.
- **Consistency Maintenance:** Ensuring LLM-generated dialogue remains consistent with pre-established character knowledge and timeline events requires careful prompt engineering and context management.
- **Cost Considerations:** Each game session incurs API costs, making the free-to-play model challenging. Estimated costs of \$0.40-1.20 per session may require monetization strategies for sustainability.
- **Network Dependency:** The application requires constant internet connectivity, limiting playability in offline scenarios and introducing potential points of failure.
- **Content Quality Variability:** While LLM outputs are generally coherent, occasional inconsistencies or implausible responses can break immersion, requiring robust validation and fallback mechanisms.

6.5 Comparison with Traditional Approaches

Aspect	Traditional Detective Games	LLM-Powered Approach
Content Creation	High upfront cost, fixed content	Low upfront cost, dynamic generation
Replayability	Limited (same story each time)	High (unique each playthrough)
Player Interaction	Predefined dialogue choices	Free-form natural language
Runtime Cost	None (offline capable)	Per-session API costs
Consistency	Guaranteed (pre-written)	Requires careful management
Response Time	Instant	5-30 second latency

6.6 Broader Implications

This project demonstrates the viability of LLM-powered interactive entertainment beyond simple chatbots. The hybrid architecture—combining structured pre-generation with dynamic on-demand content—offers a template for other narrative-driven applications such as interactive fiction, educational simulations, and personalized storytelling experiences. As LLM technology continues to improve in speed, cost, and consistency, the approach pioneered here will become increasingly practical for mainstream game development.

7 Conclusion

7.1 Project Achievements

The EE309 Detective Game successfully demonstrates the integration of Large Language Model technology with mobile game development. Key achievements include:

1. **Hybrid LLM Architecture:** Successfully implemented a balanced approach that generates complete game structures upfront (characters, locations, clues, timeline) while providing dynamic, context-aware content during gameplay (dialogues, descriptions, epilogue).
2. **Immersive Gameplay:** The combination of AI-generated mysteries, natural character dialogues, and contextual scene descriptions creates a unique, highly replayable experience where each playthrough offers a different story.
3. **Modern Android Development:** Implemented using current best practices including Jetpack Compose for declarative UI, MVVM architecture for separation of concerns, Hilt for dependency injection, and Kotlin Coroutines for asynchronous operations.

4. **Robust State Management:** The application maintains game state across configuration changes (screen rotation) and app restarts through `SavedStateHandle` and JSON serialization.
5. **Comprehensive Error Handling:** Graceful degradation when LLM calls fail, with fallback responses and clear error messaging that allows gameplay to continue.

7.2 Key Learnings

Throughout the development process, several important insights were gained:

- **LLM Integration Complexity:** Structured outputs using JSON schemas are essential for reliable parsing of LLM responses. Without strict schemas enforced via the API's `response_format` parameter, response formats can be inconsistent and cause parsing failures.
- **Prompt Engineering:** Crafting effective prompts requires significant iteration. System prompts must be highly specific about output format, tone, length constraints, and edge cases to produce consistently usable results.
- **Performance Trade-offs:** The hybrid architecture represents a deliberate trade-off between initial load time (larger upfront generation taking 30-60 seconds) and runtime responsiveness (smaller, focused API calls taking 5-10 seconds during gameplay).
- **State Management:** Complex interactive applications require careful state design. Using immutable data classes with Kotlin's `copy()` method ensures predictable state updates and simplifies debugging.

7.3 Future Work

Several enhancements could extend the project's capabilities:

1. **Save/Load System:** Full persistence using Room database for multiple save slots, allowing players to maintain multiple ongoing investigations.
2. **Offline Mode:** Caching of generated content for offline play after initial generation, reducing dependency on network connectivity.
3. **Enhanced Evidence System:** More sophisticated accusation validation requiring players to present specific evidence combinations to support their accusation.
4. **Multiplayer Support:** Cooperative investigation mode where multiple players can collaborate on solving mysteries.
5. **Voice Integration:** Speech-to-text for player questions and text-to-speech for character responses, enhancing immersion.
6. **Visual Assets:** AI-generated character portraits and location images to complement the text-based content.

7.4 Final Remarks

The EE309 Detective Game represents a successful fusion of traditional game mechanics with modern AI capabilities. By leveraging LLM technology thoughtfully—generating structure upfront and content on-demand—the application achieves both narrative consistency and dynamic storytelling. The project demonstrates that AI can significantly enhance interactive entertainment experiences when integrated with careful software engineering principles and robust error handling. The modular architecture ensures the codebase remains maintainable and extensible as LLM technology continues to advance.